ASSESSMENT TOOL 2
Case study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models**.** *(BL4)*

Assessment Tool Weightage: 40%

Code of Conduct:

I Bheeshma L (192511332) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Bheeshma L

Case study

QUESTIONS: Any 4 **Total Marks:40**

1. Online Hotel Reservation System

A travel company plans to develop an online hotel reservation platform where customers can search hotels, book rooms, make payments, and provide reviews. Hotel managers can update room availability and pricing, while administrators manage users and transactions.

Question:

Analyze the system and explain how object-oriented principles (encapsulation, inheritance, polymorphism) can be applied. Also, suggest a suitable SDLC model and justify your choice for this system.

(10 Marks)

Answer:

Application of Object-Oriented Principles: The hotel reservation platform naturally maps onto objects such as Customer, HotelManager, Administrator, Hotel, Room, Booking, Payment and Review, each combining data and behaviour.

Encapsulation: Each class hides its internal state and exposes only controlled operations. The Booking class keeps fields such as roomId, checkInDate and paymentStatus private and allows changes only through methods like confirmBooking() or cancelBooking(), so no other part of the system can corrupt booking data directly. The Payment class similarly hides card or wallet details and exposes only processPayment(), protecting sensitive financial information.

Inheritance: A common abstract class User defines shared attributes (userId, name, email, login credentials) and behaviours (login(), logout()). Customer, HotelManager and Administrator inherit from User and add role-specific attributes and methods — Customer gets searchHotel() and bookRoom(); HotelManager gets updateRoomAvailability() and updatePricing(); Administrator gets manageUsers() and monitorTransactions(). This avoids duplicating common code across roles.

Polymorphism: A method such as processTransaction() can be overridden differently for each payment mode (CreditCardPayment, UPIPayment, WalletPayment), so the checkout module calls the same method on any Payment object while the correct behaviour executes at runtime. Similarly, generateDashboard() behaves differently when invoked on a Customer, HotelManager or Administrator object.

Key Classes and Relationships: User (abstract) → Customer, HotelManager, Administrator (inheritance); Hotel "has" Room (composition, one-to-many); Customer "makes" Booking (association, one-to-many); Booking "is for" Room and "paid via" Payment; Customer

"writes" Review "for" Hotel; HotelManager "manages" Hotel; Administrator "oversees" User and Payment records.

Recommended SDLC Model: An Agile (Scrum) model is most suitable. The platform has several loosely coupled modules — search, booking, payment, reviews and administration — that map well onto sprint-based incremental delivery. Requirements such as new payment gateways, promotional offers or review-moderation rules are likely to change once real customers and hotel managers start using the system, and Agile's short sprints with continuous stakeholder feedback let the team adapt quickly. Working software can be released module by module, reducing time-to-market and risk compared with a rigid Waterfall approach.

---

2. University Management System

A university intends to automate its operations including student admissions, course registration, examination management, and result processing. Different users such as students, faculty members, examination staff, and administrators interact with the system.

Question:

Design a structured approach using object-oriented concepts to model this system. Identify key classes and relationships, and recommend an SDLC model for efficient development with reasoning.

(10 Marks)

Answer:

Structured Object-Oriented Approach: The university system can be modelled around core objects: Person, Student, Faculty, ExaminationStaff, Administrator, Course, Enrollment, Examination and Result, each encapsulating its own data and operations.

Encapsulation: The Student class stores personal and academic data (registerNumber, attendance, marks) as private fields and exposes only safe operations such as viewResult() or registerForCourse(), preventing unauthorised or accidental modification of grades. The Result class similarly encapsulates the score-computation logic behind a calculateGrade() method.

Inheritance: An abstract Person class defines common attributes (id, name, contactDetails) and behaviours (login()). Student, Faculty, ExaminationStaff and Administrator inherit from Person and extend it with role-specific attributes and methods — Student adds registerCourse() and viewResult(); Faculty adds uploadMarks(); ExaminationStaff adds scheduleExam(); Administrator adds manageUsers() and generateReports().

Polymorphism: A method such as generateReport() or viewDashboard() is overridden per role so that the same call produces an admissions summary for an Administrator, a class list for Faculty, or a personal transcript for a Student, allowing uniform interface code across very different user types.

Key Classes and Relationships: Person (abstract) → Student, Faculty, ExaminationStaff, Administrator (inheritance); Student "registers for" Course through an Enrollment association

class (many-to-many, since a student takes many courses and a course has many students); Faculty "teaches" Course; ExaminationStaff "conducts" Examination; Examination "produces" Result "for" Student; Administrator "administers" the overall system.

Recommended SDLC Model: A Spiral model is recommended. A university system is large, involves several distinct but interdependent modules (admissions, registration, examinations, results), and carries significant risk around data accuracy, academic integrity and regulatory compliance. The Spiral model's repeated cycles of planning, risk analysis, prototyping and evaluation allow each module to be built, tested with real stakeholders (students, faculty, exam staff) and refined before the next module is tackled, reducing the risk of costly errors in grade processing while still allowing requirements to evolve across the university's multi-year rollout.

---

3. Online Retail Shopping Platform

A retail company wants to develop an e-commerce platform where customers can browse products, place orders, track deliveries, and make online payments. Vendors manage product catalogs, while administrators monitor transactions and inventory.

Question:

Discuss how modular software design can be achieved using object-oriented techniques. Evaluate different SDLC models and select the most appropriate one for this scenario with justification.

(10 Marks)

Answer:

Achieving Modular Design through Object-Orientation: The retail platform is decomposed into cohesive, loosely coupled classes — Customer, Vendor, Administrator, Product, Cart, Order, Payment and DeliveryTracking — each responsible for a single concern and communicating through well-defined interfaces. This modularity lets the catalog, cart, payment and delivery-tracking subsystems be developed, tested and scaled independently.

Encapsulation: The Product class hides stock and pricing logic behind methods such as updateStock() and applyDiscount(), so inventory can only be changed through validated operations, not by directly editing fields from other modules. The Order class similarly encapsulates order-status transitions behind methods like placeOrder() and cancelOrder().

Inheritance: An abstract User class is extended by Customer, Vendor and Administrator, sharing login and profile attributes while adding role-specific behaviour (Customer: browseProducts(), placeOrder(); Vendor: manageCatalog(); Administrator: monitorTransactions()). An abstract Payment class is similarly extended by CreditCardPayment, WalletPayment and CashOnDelivery.

Polymorphism: Every Payment subclass overrides processPayment(), so the checkout module can call processPayment() on any payment object without knowing its concrete type, and new

payment methods can be added later simply by adding a new subclass — an example of the open/closed principle enabled by polymorphism — without altering existing checkout code.

SDLC Evaluation and Recommendation: Waterfall is unsuitable because e-commerce requirements (new payment gateways, promotions, delivery partners) change frequently and Waterfall cannot easily accommodate late changes. A pure Spiral model would add unnecessary process overhead for a business-driven web platform. An Agile (Scrum) model is the best fit: it delivers features in short, testable increments (catalog browsing first, then cart and checkout, then delivery tracking), incorporates continuous feedback from customers, vendors and administrators, and matches the loosely coupled, object-oriented module structure, letting the platform evolve quickly to stay competitive.

---

4. Smart Waste Management System

A municipality introduces a smart waste management system where IoT-enabled bins monitor waste levels and automatically notify collection vehicles when bins are full. Citizens can also report waste-related issues through a mobile application.

Question:

Apply object-oriented system development principles to design this system. Identify the life cycle model best suited for handling real-time data, sensor integration, and evolving requirements, and explain why.

(10 Marks)

Answer:

Object-Oriented System Design: The smart waste management system can be modelled with classes such as Device (abstract), WasteBin, CollectionVehicle, Citizen, IssueReport, Sensor, Route and NotificationService, capturing both the physical IoT devices and the software components that process their data.

Encapsulation: The WasteBin class hides raw sensor readings and communication details behind private fields (fillLevel, batteryStatus) and exposes only safe methods such as getStatus() or reportFull(), so other modules never interact with the sensor hardware directly. This isolates hardware-specific logic from the rest of the system.

Inheritance: An abstract Device class defines common attributes (deviceId, location, status) and behaviours (sendAlert()). WasteBin and CollectionVehicle inherit from Device and add specialised behaviour — WasteBin adds monitorFillLevel(); CollectionVehicle adds updateRoute(). A common IssueReport hierarchy can likewise distinguish citizen-reported issues by type.

Polymorphism: The sendAlert() / notify() method is overridden differently across device and user types: a WasteBin triggers a “bin full” alert to a CollectionVehicle, while a Citizen’s IssueReport triggers a maintenance alert to Administrators. The system can invoke notify() uniformly without needing to know the concrete object type, simplifying the notification service.

Recommended Life Cycle Model: A Spiral model, ideally combined with Agile iterations within each spiral cycle, is best suited here. The system integrates hardware (IoT sensors, communication modules) with real-time software, so each spiral cycle can prototype and test one risk area — sensor reliability, network latency, alert accuracy — before committing to full-scale development. This risk-driven, iterative approach handles the technical uncertainty of sensor integration and real-time data processing far better than a linear model, while short Agile-style iterations within each spiral loop keep the system adaptable to evolving municipal requirements (new sensor types, additional citizen-reporting features, changing collection policies).

5. Smart Healthcare Management System

A multi-specialty hospital plans to develop a Smart Healthcare Management System where patients can book appointments, consult doctors, access medical records, and make online payments. Doctors update patient diagnoses and prescriptions, while hospital administrators manage departments, staff, and billing information.

Question:

Analyze the system and explain how **object-oriented principles (encapsulation, inheritance, and polymorphism)** can be applied. Also, recommend a suitable **SDLC model** for developing this healthcare system and justify your choice.

(10 Marks)

---

6. Smart Library Management System

A university library intends to automate its operations, allowing students and faculty members to search books, borrow and return books, reserve titles online, and pay overdue fines. Librarians manage book inventories, memberships, and circulation records, while administrators generate reports and maintain the system.

Question:

Design a structured solution using **object-oriented concepts** for this system. Identify the major classes, attributes, methods, and relationships, and recommend an appropriate **SDLC model** for efficient development with suitable justification.

(10 Marks)